

Le operazioni bit a bit (bitwise)

Operatori bitwise

Sono operatori che si possono usare su variabili di tipo char o int (unsigned) manipolando il dato a livello di singolo bit.

Le operazioni bit a bit

- NOT bit a bit $\sim$
- AND bit a bit $\&$
- OR bit a bit $|$
- XOR bit a bit $\wedge$
- SHIFT A DESTRA $>>$
- SHIFT A SINISTRA $<<$

Categorie

Possono essere raggruppati in tre categorie:

- Operatori logici bit-a-bit (and, or, xor)
- Operatore di complemento bit-a-bit (not)
- Operatori di shift (traslazione) (<<,>>)

Differenza tra op. logici e op. bit a bit

Gli operatori bit a bit `&`, `|` non vanno confusi con gli operatori logici `&&`, `||`

```
int a = 1;
int b = 2;

int ex1 = a && b;    // vale 1 (true)
int ex2 = a & b;     // vale 0
```

AND - &

Setta ad 1 un bit in una data posizione quando **entrambi** i bit in quella posizione nei due operandi valgono 1, altrimenti lo setta a 0.

a=63;	0 0 1 1 1 1 1 1	
b=240;	1 1 1 1 0 0 0 0	
c = a & b;	0 0 1 1 0 0 0 0	c=48

OR - |

Setta ad 1 un bit in una data posizione quando **almeno uno** dei bit in quella posizione nei due operandi vale 1 altrimenti lo setta a 0.

a=60

0 0 1 1 1 1 0 0

b=240

1 1 1 1 0 0 0 0

c =a | b;

1 1 1 1 1 1 0 0 c=252

Xor - ^

Setta ad 1 un bit in una data posizione quando i bit in quella posizione nei due operandi **sono diversi**, altrimenti lo setta a 0.

s=60	0 0 1 1 1 1 0 0	
b=240	1 1 1 1 0 0 0 0	
c = a ^ b	1 1 0 0 1 1 0 0	c=204

Not ~

Setta ad 1 i bit che valgono 0 e setta a 0 i bit che valgono 1.

a=60; 0 0 1 1 1 1 0 0

b = ~ a; 1 1 0 0 0 0 1 1

b = 195

Shift - >> <<

Lo shift a destra o a sinistra sposta tutti i bit di una o più posizioni a destra o a sinistra

SHIFT a sinistra

int a = 74; 0 1 0 0 1 0 1 0

int b = a << 2; 0 0 1 0 1 0 0 0 risultato b=296

SHIFT a destra

int a = 74; 0 1 0 0 1 0 1 0

int b = a >> 2; 0 0 0 1 0 0 1 0 risultato b=18

Lo shift a destra equivale a una divisione per multipli di 2 (nell'esempio $\gg 2$ è una divisione per 4), mentre lo shift a sinistra equivale a una moltiplicazione per multipli di 2 ($\ll 2$ è una moltiplicazione per 4).

Esercizio - 13/4/2022

1) Provare le operazioni bit a bit con a=21 e b=54

```
int a = 21;
int b = 54;
int c = a & b;
int d = a | b;
int e = a ^ b;
int f = ~a;
int g = a << 3;
int h = a >> 2;

printf("a = %d\n", a);
printf("b = %d\n", b);
printf("a & b = %d\n", c);
printf("a | b = %d\n", d);
printf("a ^ b = %d\n", e);
printf("~a = %d\n", f);
printf("a << 3 = %d\n", g);
printf("a >> 2 = %d\n", h);
```

Attenzione

$a \& b == 5$ -----> $a \& (b == 5)$

a differenza di

$a + b == 5$ -----> $(a + b) == 5$

Quindi per sicurezza usare sempre le parentesi

Esercizio 13/4/2022

- 2 - Scrivere un programma che stampa il valore di ogni bit di una variabile di tipo char
- 3 – Scrivere un programma che conta i bit ad 1

Le maschere

Una MASCHERA è un numero binario che permette di vedere solo alcuni bit quelli che servono all'interno di un valore.

Esempio:

$X = \text{valore}$

$\text{maschera} = 15 \text{ (00001111)}$

$X \text{ AND maschera}$ dà come risultato i quattro bit bassi qualsiasi sia il valore.

Questo equivale all'operazione di mod 16.

Esercizio 13/4/2022

4) Chiedere in input un intero da 0 a 255 (max rappresentabile con 8 bit), controlla l'input e stampare il numero in binario. Calcolare la parità p (pari) e aggiungerla e al numero inserito che diventerà n_con_p . Stampare in binario il numero con il bit di parità n_con_p (ricorda che il bit di parità va aggiunto in testa, quindi è il bit più significativo).

Esercizi 13/4/2022

5) Chiedere in input un intero n da 0 a 255 (max rappresentabile con 8 bit), controllare l'input e stampa il numero in binario.

Calcolare la parità $p1$ e aggiungerla al numero inserito che diventerà n_con_p1 .

Stampare in binario il numero con il bit di parità n_con_p1 : è questo il numero che verrà trasmesso.

Creare una maschera $mask$ con un solo bit 1 e tutti gli altri a 0 (2^{pos} oppure $1 \ll pos$, dove pos è un numero casuale fra 0 e 8). Stampare la maschera ottenuta in binario.

Simulare un errore di trasmissione, cambiando il valore di un bit del numero n_con_p1 . Per fare ciò utilizzare la maschera $mask$ creata ed effettuare operazione XOR col n_con_p1 .

Stampare in binario il numero corrotto ottenuto n_err

Calcolare la parità di n_err , $p2$

Rilevare o meno la presenza dell'errore (calcolando la parità $p2$ di n_err otterremo 1 o 0: se $p2 = 1$ il numero è arrivato corrotto a destinazione, altrimenti è integro)

Parità incrociata e distanza di Hamming

Parità longitudinale

La parità longitudinale consiste nell'aggiungere un byte alla fine della trasmissione di un blocco di n dati. Questo byte è chiamato **byte di checksum** e viene calcolato eseguendo l'operazione di XOR su tutti i byte del blocco trasmesso.

Es.

0 0 1 1 0 0 1 1 ^

0 1 0 1 0 1 1 0 ^

0 0 0 1 1 1 1 0 ^

0 0 0 0 0 1 1 1 ^

0 1 1 1 1 1 0 0

Parità incrociata

La parità incrociata è una tecnica per la correzione degli errori che consiste nel calcolare oltre al bit di parità per ogni byte (parità trasversale) anche un bit di parità per i bit che occupano la stessa posizione in ogni byte di un blocco (parità longitudinale).

Esempio:

Parità trasversale

1 0 1 1 0 0 1 1

0 1 0 1 0 1 1 0

0 0 0 1 1 1 1 0

1 0 0 0 0 1 1 1

0 1 1 1 1 1 0 0

Byte per parità longitudinale

Domande

Se trasmetto un pacchetto di byte con la parità trasversale:

- 1) Sono in grado di rilevare gli errori? Non sempre. Rilevo solo l'esistenza di errori se il numero di bit errati è dispari.
- 2) Se ho rilevato errore, sono in grado di capire quali sono i bit errati? No.

Se trasmetto un pacchetto di byte con la parità longitudinale:

- 1) Sono in grado di rilevare errori? Non sempre. Rilevo solo l'esistenza di errori se il numero di bit errati è dispari.
- 2) Sono anche in grado di capire quali sono i byte danneggiati? No

Se trasmetto un pacchetto con parità trasversale e longitudinale (incrociata):

- 1) Sono in grado di rilevare errori? Sì (non tutti)
- 2) Sono anche in grado di capire quali sono i byte danneggiati? Solo se vi è uno solo byte corrotto e riesco ad identificarlo con la parità trasversale. In tal caso riesco anche a correggerlo.

Parità incrociata

```
11001100 0
01000111 0
11011000 0
11110000 0
10101101 1
01001001 1
00001011 1
11110111 1
-----
10111011
```

Se si dovesse verificare un errore su un bit, si potrebbe determinare la posizione del bit errato all'incrocio dei bit di parità che risultano diversi.

```
11001100 0
01000101 1
11011000 0
11110000 0
10101101 1
01001001 1
00001011 1
11110111 1
-----
10111001
```

Esercizio parità 20/4/2022

Scrivere un programma che generi 8 numeri di 8 bit casuali (tra 0 e 255),

- 1) utilizzando una funzione `paritaTrasversale`, aggiungere ad ognuno degli otto numeri calcolati la sua parità e porli in un array di 9 interi.
- 2) Utilizzando una funzione `paritaLongitudinale` aggiungere nella nona cella dell'array la parità longitudinale.
- 3) Stampare l'array in binario utilizzando una funzione